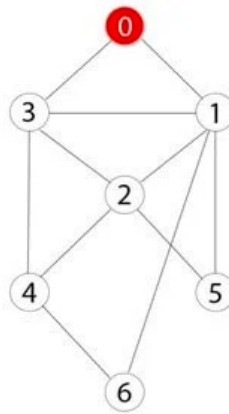


Activity- 1:

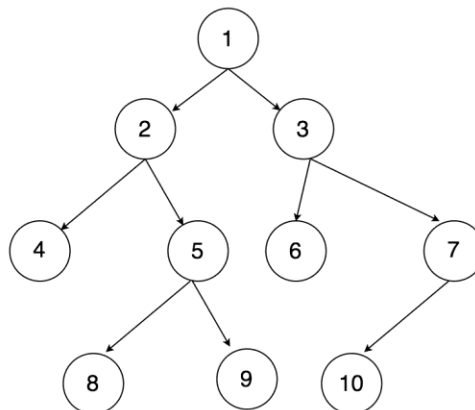
Consider the following graph. If there is ever a decision between multiple neighbor nodes in the BFS algorithm, assume we always choose the letter closest to the beginning of the alphabet first. A connected graph with 7 nodes and 7 edges. The edges are undirected and unweight. Distance between two nodes will be measured based on the number of edges separating two vertices. Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors.



Define function name `_connected_component`, this function keep track of all the visited nodes with BFS, is as simple as implementing the steps of the algorithm and assign `_queue` variable already has a node to be checked, i.e., the starting vertex that is used as an entry point to explore the graph. The next step is to implement a loop that keeps cycling until queue is empty. At each iteration of the loop, a node is checked. If this wasn't visited already, its neighbors are added to queue. Once the loop is exited, the function (`connected_component`) returns all of the visited nodes.

Activity- 2:

Apply Breadth First Search on following graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.



Activity- 3:

Repeat Activity-2, apply BFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

Activity- 4:

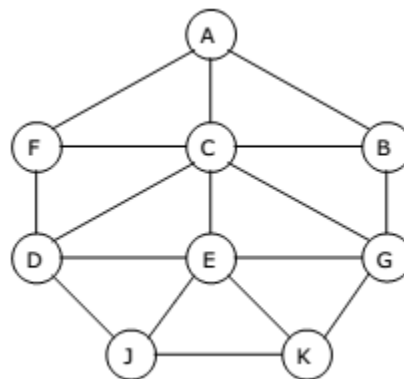
Apply Depth First Search on Activity-2 graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.

Activity- 5:

Repeat Activity-4, apply DFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

Activity- 6:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Breadth First Search on following graph considering the initial state is A and final state is K. Also evaluate it manually.

**Activity- 7:**

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Depth First Search on Activity-6 graph considering the initial state is A and final state is K. Also evaluate it manually.